

Experimental Evaluation of the Hadoop Ecosystem Performance in Docker and Virtual Machine Environments for Distributed Data Processing

Evaluación experimental del rendimiento del ecosistema Hadoop en entornos Docker y máquinas virtuales para el procesamiento distribuido de datos

Andrea Juliet Reyna Saravia¹

¹Universidad Nacional de Cañete, Lima, Perú. Correo electrónico: 2301080272@undc.edu.pe

Recibido:

Aceptado:

Publicado:



Copyright: © 2026 los autores.

Este trabajo es un artículo de acceso abierto

Abstract

Background: The massive increment of data within the automotive industry demands highly efficient distributed processing frameworks. This paper presents an experimental and comparative performance analysis between Hadoop MapReduce and Apache Spark when executing complex analytical aggregations on commercial data

Methods: Through an experimental and quantitative approach, a unified distributed infrastructure consisting of six Docker containers organized into independent interconnected clusters was deployed, with resources orchestrated by YARN. Two scaled variants of a historical dataset from the Illinois automotive market (2018–2020), sized at ~200 MB and ~1 GB, were preloaded into HDFS and processed. Benchmarking evaluated an analytical query measuring six business metrics per vehicle brand, independently coded via Hadoop Streaming (Python) and PySpark DataFrames. Resource monitoring was automated using the YARN ResourceManager REST API, and the collected metadata was integrated into interactive Power BI Desktop dashboards.

Results: YARN metrics indicated that Hadoop MapReduce achieved faster processing times in both scenarios, outperforming Apache Spark by 47.5% in the small dataset (35.5 s vs. 67.6 s) and by 50.0% in the large dataset (47.6 s vs. 95.2 s) due to Spark's initial session initialization overhead. However, when scaling from 200 MB to 1 GB, Spark demonstrated superior hardware scalability; its memory and CPU consumption grew by only 52.6% and 51.2% respectively, whereas MapReduce exhibited drastic increments of 172.8% in memory and 213.3% in VCore-seconds. Notably, YARN infrastructure allocation remained constant at exactly one registered container for all runs.

Conclusions: A comprehensive benchmarking and data visualization pipeline was successfully validated. It is concluded that Hadoop MapReduce provides higher time efficiency for single-pass transactions on moderate datasets due to negligible initialization costs, although its hardware demands degrade steeply when scaling. Conversely, Apache Spark scales more efficiently in terms of memory and CPU, mathematically confirming its suitability for iterative workflows or massive scales of data. Finally, decoupling analytical and infrastructural metadata into Power BI establishes a practical framework for Big Data computing cluster auditing.

Keywords: Big Data, Hadoop MapReduce, Apache Spark, Distributed processing, Automotive market, Power BI.

Resumen

Antecedentes: El incremento masivo de datos en la industria automotriz exige frameworks de procesamiento distribuido altamente eficientes. Este artículo presenta un análisis comparativo y experimental del rendimiento computacional entre Hadoop MapReduce y Apache Spark al ejecutar agregaciones complejas sobre datos comerciales.

Métodos: Mediante un enfoque experimental cuantitativo, se implementó una infraestructura distribuida unificada compuesta por seis contenedores Docker organizados e interconectados en clústeres independientes, cuya asignación de recursos analíticos fue orquestada por YARN. Se procesaron dos variantes a escala de un dataset histórico del mercado automotriz de Illinois (2018–2020) con volúmenes de ~200 MB y ~1 GB precargados en HDFS. El experimento (*benchmarking*) evaluó una consulta analítica de seis métricas de negocio por marca vehicular, codificada de manera independiente con Hadoop Streaming (Python) y PySpark DataFrames. El monitoreo de recursos se automatizó mediante la

Resultados: Las métricas de YARN demostraron que Hadoop MapReduce superó a Apache Spark en velocidad en ambos escenarios, siendo un 47.5% más rápido en el dataset pequeño (35.5 s vs. 67.6 s) y un 50.0% más rápido en el dataset grande (47.6 s vs. 95.2 s) debido al costo inicial de arranque (*overhead*) de Spark. No obstante, al escalar de 200 MB a 1 GB, Spark demostró una estabilidad superior en hardware; su consumo de memoria y CPU creció únicamente un 52.6% y un 51.2% respectivamente, mientras que MapReduce exhibió incrementos drásticos del 172.8% en memoria y un 213.3% en VCore-segundos. La asignación de infraestructura de YARN se mantuvo fija en un único contenedor registrado para todas las ejecuciones.

Conclusiones: Se validó con éxito un flujo completo de *benchmarking* y visualización de datos. Se concluye que Hadoop MapReduce proporciona mayor eficiencia en tiempo para cargas de trabajo moderadas de una sola pasada debido a su nulo costo de inicialización, aunque su consumo de hardware se degrada pronunciadamente al escalar. En contraparte, Apache Spark escala de manera más eficiente en memoria y CPU, lo que confirma matemáticamente su idoneidad para flujos de trabajo iterativos o conjuntos de datos de mayor envergadura. Finalmente, el desacoplamiento de metadatos analíticos y de infraestructura hacia Power BI establece un paradigma práctico para la auditoría de clústeres tecnológicos de Big Data.

Palabras claves: Big Data, Hadoop MapReduce, Apache Spark, Procesamiento distribuido, Mercado automotriz, Power BI.

Abstract

Background: The massive increment of data within the automotive industry demands highly efficient distributed processing frameworks. This paper presents an experimental and comparative performance analysis between Hadoop MapReduce and Apache Spark when executing complex analytical aggregations on commercial data.

Methods: Through an experimental and quantitative approach, a unified distributed infrastructure consisting of six Docker containers organized into independent interconnected clusters was deployed, with resources orchestrated by YARN. Two scaled variants of a historical dataset from the Illinois automotive market (2018–2020), sized at ~200 MB and ~1 GB, were preloaded into HDFS and processed. Benchmarking evaluated an analytical query measuring six business metrics per vehicle brand, independently coded via Hadoop Streaming (Python) and PySpark DataFrames. Resource monitoring was automated using the YARN ResourceManager REST API, and the collected metadata was integrated into interactive Power BI Desktop dashboards.

Results: YARN metrics indicated that Hadoop MapReduce achieved faster processing times in both scenarios, outperforming Apache Spark by 47.5% in the small dataset (35.5 s vs. 67.6 s) and by 50.0% in the large dataset (47.6 s vs. 95.2 s) due to Spark's initial session initialization overhead. However, when scaling from 200 MB to 1 GB, Spark demonstrated superior hardware scalability; its memory and CPU consumption grew by only 52.6% and 51.2% respectively, whereas MapReduce exhibited drastic increments of 172.8% in memory and 213.3% in VCore-seconds. Notably, YARN infrastructure allocation remained constant at exactly one registered container for all runs.

Conclusions: A comprehensive benchmarking and data visualization pipeline was successfully validated. It is concluded that Hadoop MapReduce provides higher time efficiency for single-pass transactions on moderate datasets due to negligible initialization costs, although its hardware demands degrade steeply when scaling. Conversely, Apache Spark scales more efficiently in terms of memory and CPU, mathematically confirming its suitability for iterative workflows or massive scales of data. Finally, decoupling analytical and infrastructural metadata into Power BI establishes a practical framework for Big Data computing cluster auditing.

Keywords: Big Data, Hadoop MapReduce, Apache Spark, Distributed processing, Automotive market, Power BI.

1 INTRODUCCIÓN

El crecimiento exponencial de los datos generados por organizaciones, aplicaciones digitales y sistemas distribuidos ha impulsado el desarrollo de plataformas capaces de procesar grandes volúmenes de información de manera eficiente. En este contexto, las tecnologías de Big Data han adquirido gran importancia debido a su capacidad para administrar datos a gran escala mediante arquitecturas distribuidas. Entre estas plataformas, Apache Hadoop se ha consolidado como uno de los ecosistemas más utilizados para el procesamiento paralelo, empleando componentes como Hadoop Distributed File System (HDFS) y MapReduce para distribuir cargas computacionales entre múltiples nodos [5], [7].

Sin embargo, el rendimiento de Hadoop depende directamente de la infraestructura donde es desplegado, debido a factores como la administración de recursos, la planificación de tareas, la latencia de procesamiento y la eficiencia del almacenamiento. Diversos estudios han abordado la optimización de Hadoop MapReduce mediante mejoras en la programación de tareas y la adaptación del procesamiento en clústeres heterogéneos, demostrando que la configuración del entorno computacional influye significativamente en la eficiencia del sistema [6], [8]. Asimismo, los mecanismos de detección y recuperación de fallos continúan siendo aspectos fundamentales para garantizar la confiabilidad de los sistemas Hadoop distribuidos [9].

En este escenario, la virtualización representa una alternativa ampliamente utilizada para implementar infraestructuras Big Data debido a su capacidad de aislamiento y administración flexible de recursos. Las máquinas virtuales proporcionan entornos completos de ejecución, aunque pueden introducir una mayor sobrecarga debido a la virtualización del hardware. Por otro lado, los contenedores Docker ofrecen un enfoque más ligero al compartir el núcleo del sistema operativo anfitrión, permitiendo reducir el consumo de recursos y facilitar el despliegue de aplicaciones distribuidas [1], [3], [4]. Investigaciones previas han demostrado que Docker puede mejorar la portabilidad y administración de clústeres Hadoop, aunque el rendimiento puede variar según la carga de trabajo y la configuración del entorno [2].

Por ello, el presente trabajo propone realizar una evaluación experimental del rendimiento del ecosistema Hadoop en entornos Docker y máquinas virtuales para el procesamiento distribuido de datos. Mediante la implementación de escenarios controlados, se analizarán métricas de desempeño que permitan comparar ambas tecnologías de virtualización considerando aspectos como tiempo de ejecución, utilización de recursos computacionales y eficiencia del procesamiento. Además, se tendrá en cuenta la importancia de la gestión de almacenamiento y operaciones de entrada/salida en sistemas Big Data contenerizados, debido a su impacto directo en el rendimiento global de la plataforma [10]. Los resultados obtenidos permitirán determinar las ventajas y limitaciones de cada entorno de despliegue, proporcionando criterios técnicos para la selección de arquitecturas orientadas a aplicaciones de procesamiento masivo de datos.

2 MÉTODO

En esta sección se describe la metodología experimental utilizada para evaluar el rendimiento del ecosistema Hadoop en entornos basados en contenedores Docker y máquinas virtuales. El estudio considera la implementación de dos escenarios de despliegue equivalentes, manteniendo condiciones similares de hardware, configuración de red y recursos asignados, con el propósito de comparar el comportamiento de Hadoop bajo diferentes tecnologías de virtualización.

La evaluación se basa en la ejecución de cargas de trabajo distribuidas mediante Hadoop MapReduce, considerando métricas relacionadas con el tiempo de procesamiento, consumo de recursos computacionales y eficiencia del sistema. La metodología está compuesta por cuatro etapas principales:

2.1 Diseño del entorno experimental

El entorno experimental fue diseñado con el objetivo de evaluar comparativamente el rendimiento del ecosistema Hadoop bajo dos escenarios de despliegue: contenedores Docker y máquinas virtuales. Para ello, se plantea una arquitectura distribuida equivalente en ambos ambientes, compuesta por un clúster Hadoop de tres nodos: un nodo maestro encargado de ejecutar los servicios NameNode y ResourceManager, y dos nodos trabajadores responsables de ejecutar los servicios DataNode y NodeManager. Ambos escenarios utilizarán la misma configuración de software, incluyendo Apache Hadoop y Java, así como una asignación equivalente de recursos computacionales, permitiendo analizar únicamente el impacto generado por la tecnología de virtualización empleada. El entorno basado en Docker será administrado mediante Visual Studio Code, mientras que el escenario de máquinas virtuales será implementado sobre CentOS, proporcionando una base experimental homogénea para la ejecución de procesos distribuidos y la posterior comparación de métricas de rendimiento.

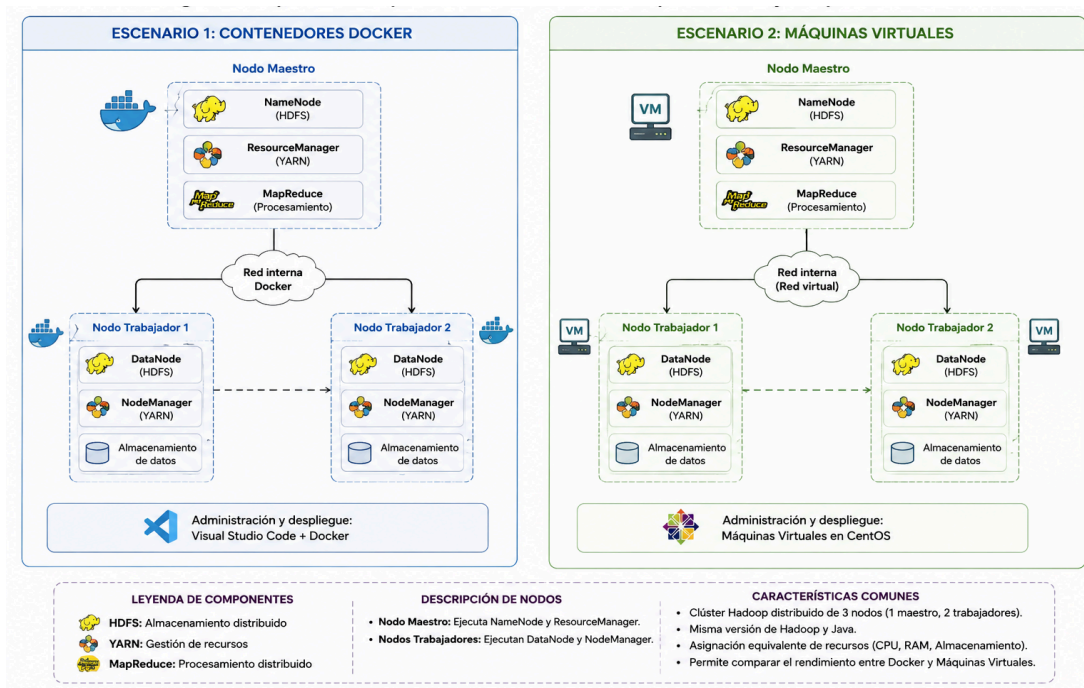


Fig. 1. Arquitectura experimental del clúster Hadoop en Docker y máquinas virtuales. ahora esta imagen

2.2 Implementación del ecosistema Hadoop

Sobre la arquitectura experimental definida, se implementó el ecosistema Hadoop en ambos escenarios de despliegue mediante los componentes HDFS, YARN y MapReduce. HDFS fue configurado como sistema de almacenamiento distribuido, permitiendo gestionar la distribución de bloques de datos entre los nodos trabajadores. El servicio NameNode fue encargado de administrar los metadatos del sistema de archivos, mientras que los servicios DataNode almacenaron físicamente la información distribuida.

Asimismo, se configuró YARN como administrador de recursos del clúster, asignando el servicio ResourceManager para la coordinación de recursos y NodeManager para la gestión de tareas ejecutadas en cada nodo trabajador. Además, se habilitó el modelo de procesamiento MapReduce para la ejecución de cargas distribuidas sobre los datos almacenados en HDFS. La comunicación entre los servicios Hadoop fue establecida mediante una red interna entre los nodos del clúster, garantizando la interacción entre los componentes distribuidos. Esta implementación permitió disponer de un entorno Hadoop funcional en Docker y máquinas virtuales para la posterior evaluación de rendimiento.

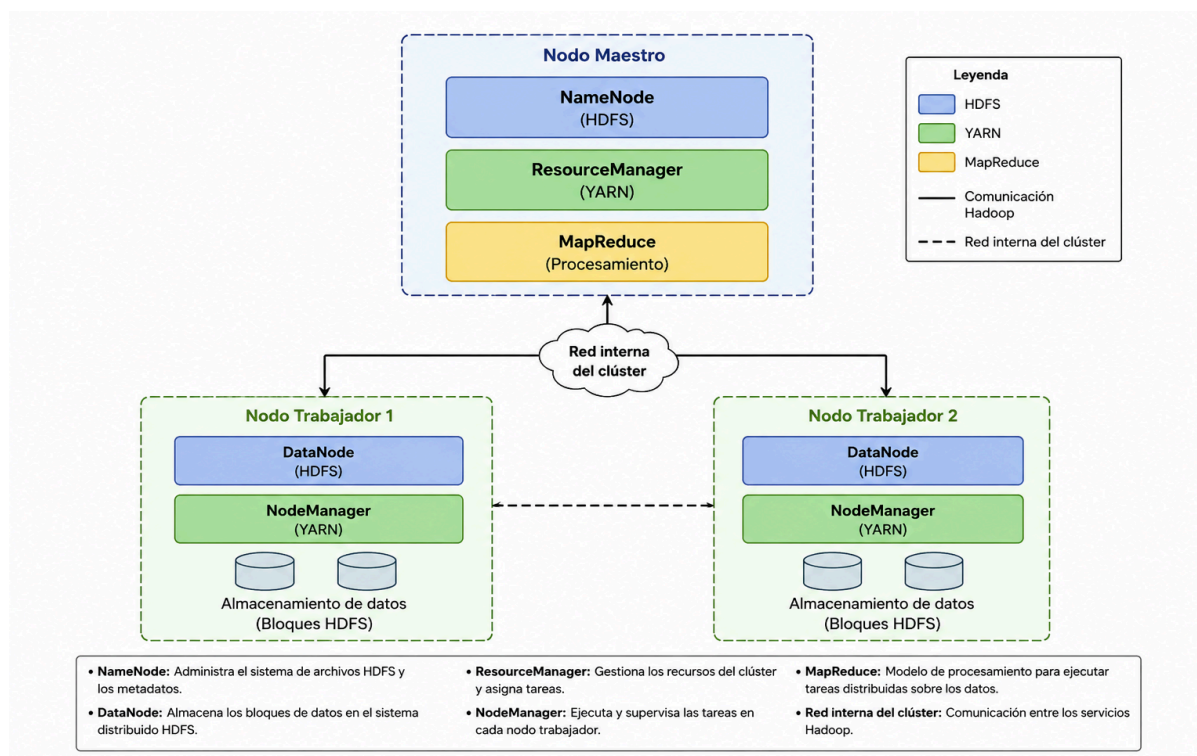


Fig. 2. Arquitectura del ecosistema Hadoop implementado en ambos escenarios experimentales

2.3 Configuración de escenarios Docker y Máquina Virtual

Luego de implementar el ecosistema Hadoop en ambos ambientes experimentales, se procedió a configurar los escenarios de despliegue basados en Docker y máquinas virtuales. Esta etapa tuvo como finalidad establecer las condiciones específicas de cada tecnología de virtualización, considerando aspectos relacionados con la creación de instancias, comunicación entre nodos y administración de recursos computacionales. Ambos escenarios conservaron la misma arquitectura distribuida definida previamente, permitiendo realizar una comparación objetiva del rendimiento del ecosistema Hadoop bajo diferentes mecanismos de despliegue.

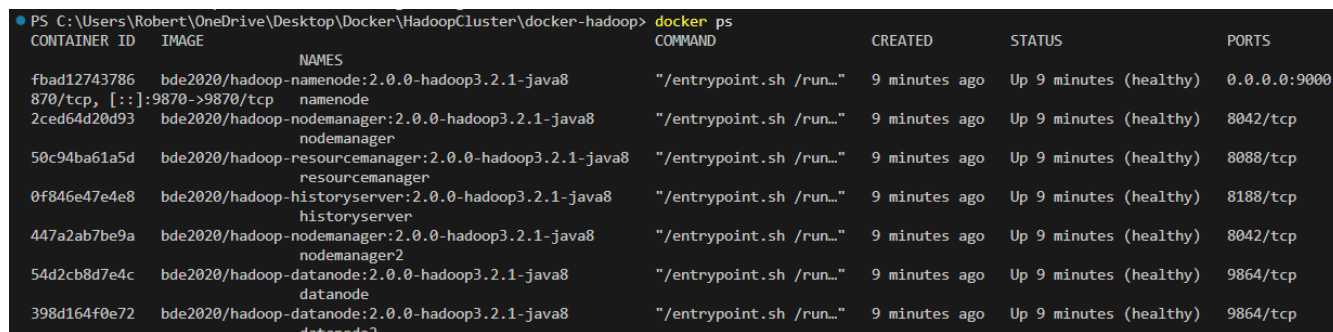
2.3.1 Escenario basado en Docker

El escenario basado en Docker fue configurado mediante contenedores utilizando Docker Compose como herramienta de orquestación. Para ello, se emplearon imágenes preconfiguradas del ecosistema Hadoop, a partir de las cuales se generaron los servicios correspondientes al clúster distribuido.

La implementación estuvo compuesta por un conjunto de contenedores conectados mediante una red interna de Docker, donde cada servicio Hadoop fue ejecutado de manera independiente. El nodo maestro fue desplegado mediante los contenedores correspondientes a los servicios principales del clúster, mientras que los nodos trabajadores fueron configurados mediante contenedores adicionales encargados de proporcionar capacidad de almacenamiento y procesamiento distribuido.

La comunicación entre los contenedores se realizó mediante la red virtual creada por Docker Compose, permitiendo la interacción entre los diferentes servicios sin requerir configuraciones externas de red. Asimismo, los recursos computacionales asignados a los contenedores fueron gestionados mediante el motor Docker, manteniendo un entorno ligero al compartir el sistema operativo del equipo anfitrión.

La correcta configuración del escenario fue validada mediante la ejecución de los contenedores activos y la comprobación del registro de los nodos trabajadores dentro del clúster Hadoop.



CONTAINER ID	IMAGE	NAMES	COMMAND	CREATED	STATUS	PORTS
fbad12743786	bde2020/hadoop-namenode:2.0.0-hadoop3.2.1-java8	namenode	"/entrypoint.sh /run..."	9 minutes ago	Up 9 minutes (healthy)	0.0.0.0:9000
870/tcp, [::]:9870->9870/tcp						
2ced64d20d93	bde2020/hadoop-nodemanager:2.0.0-hadoop3.2.1-java8	nodemanager	"/entrypoint.sh /run..."	9 minutes ago	Up 9 minutes (healthy)	8042/tcp
50c94ba61a5d	bde2020/hadoop-resourcemanager:2.0.0-hadoop3.2.1-java8	resourcemanager	"/entrypoint.sh /run..."	9 minutes ago	Up 9 minutes (healthy)	8088/tcp
0f846e47e4e8	bde2020/hadoop-historyserver:2.0.0-hadoop3.2.1-java8	historyserver	"/entrypoint.sh /run..."	9 minutes ago	Up 9 minutes (healthy)	8188/tcp
447a2ab7be9a	bde2020/hadoop-nodemanager:2.0.0-hadoop3.2.1-java8	nodemanager2	"/entrypoint.sh /run..."	9 minutes ago	Up 9 minutes (healthy)	8042/tcp
54d2cb8d7e4c	bde2020/hadoop-datanode:2.0.0-hadoop3.2.1-java8	datanode	"/entrypoint.sh /run..."	9 minutes ago	Up 9 minutes (healthy)	9864/tcp
398d164f0e72	bde2020/hadoop-datanode:2.0.0-hadoop3.2.1-java8	datanode2	"/entrypoint.sh /run..."	9 minutes ago	Up 9 minutes (healthy)	9864/tcp

Fig. 3. Contenedores Hadoop desplegados en el escenario basado en Docker

2.3.2 Escenario basado en máquinas virtuales

El escenario basado en máquinas virtuales fue configurado mediante instancias virtualizadas independientes, donde cada máquina representó un nodo dentro del clúster Hadoop distribuido. A diferencia del escenario Docker, cada nodo virtual dispuso de un sistema operativo propio, recursos asignados individualmente y una configuración independiente de red.

Para este escenario se implementó una máquina virtual destinada al nodo maestro y dos máquinas virtuales correspondientes a los nodos trabajadores. La configuración de cada instancia incluyó la instalación del sistema operativo CentOS, la configuración de los servicios Hadoop necesarios y la asignación de recursos de procesamiento y memoria para permitir la ejecución de tareas distribuidas.

La comunicación entre las máquinas virtuales fue establecida mediante una red virtual interna, permitiendo la transferencia de información entre los nodos y la ejecución coordinada de procesos Hadoop. La disponibilidad de los servicios fue verificada mediante la herramienta de monitoreo de procesos Java (jps), comprobando la ejecución correcta de los componentes del clúster en cada nodo.

```
[reyna3@master ~]$ start-dfs.sh
Starting namenodes on [master]
Starting datanodes
Starting secondary namenodes [master]
[reyna3@master ~]$ start-yarn.sh
Starting resourcemanager
Starting nodemanagers
[reyna3@master ~]$ jps
4640 ResourceManager
4183 SecondaryNameNode
3864 NameNode
4942 Jps
```

Fig. 4. Procesos Hadoop ejecutándose en el escenario basado en máquinas virtuales

Tabla 1. Configuración de los escenarios experimentales

Característica	Docker	Máquina virtual
Tecnología de despliegue	Contenedores Docker	Virtualización completa
Administración del entorno	Docker Compose	Hipervisor de máquinas virtuales
Sistema operativo	Compartido con el equipo anfitrión	Independiente por nodo
Arquitectura Hadoop	1 maestro + 2 trabajadores	1 maestro + 2 trabajadores
Comunicación	Red interna Docker	Red virtual entre máquinas
Asignación de recursos	Recursos gestionados por Docker	Recursos asignados a cada máquina virtual

2.4 Diseño de las pruebas de rendimiento

Luego de establecer los escenarios experimentales basados en Docker y máquinas virtuales, se diseñó un conjunto de pruebas de rendimiento orientadas a evaluar el comportamiento del ecosistema Hadoop bajo diferentes condiciones de carga y frameworks de procesamiento distribuido. El diseño experimental tuvo como finalidad obtener información comparable sobre el desempeño del clúster en ambos ambientes de despliegue, manteniendo las mismas condiciones de ejecución para cada prueba.

Las pruebas fueron realizadas considerando como variables experimentales el ambiente de despliegue, el framework de procesamiento utilizado y el volumen de datos procesado. Los escenarios evaluados corresponden

al clúster Hadoop desplegado mediante contenedores Docker y al clúster Hadoop implementado sobre máquinas virtuales con sistema operativo CentOS.

Para el procesamiento distribuido se utilizaron dos frameworks pertenecientes al ecosistema Hadoop: Hadoop MapReduce y Apache Spark. La inclusión de ambos frameworks permitió analizar el comportamiento del sistema utilizando un modelo tradicional basado en procesamiento por etapas y un modelo optimizado orientado al procesamiento en memoria.

Con el objetivo de evaluar la escalabilidad del sistema ante diferentes tamaños de información, se utilizaron tres datasets con volúmenes progresivos. Cada conjunto de datos fue almacenado previamente en el sistema distribuido de archivos HDFS y posteriormente procesado bajo las mismas condiciones experimentales en ambos escenarios.

Los datasets utilizados fueron:

- **ventas_200MB.csv:** utilizado para representar una carga de trabajo pequeña y analizar el comportamiento inicial del clúster.
- **ventas_1GB.csv:** empleado para evaluar el rendimiento ante un volumen intermedio de información.
- **dataset_5GB.csv:** utilizado para analizar la capacidad de procesamiento distribuido frente a una carga mayor de datos.

Tabla 2. Diseño experimental de las pruebas de rendimiento

Variable experimental	Configuración
Escenario experimental 1	Hadoop sobre Docker
Escenario experimental 2	Hadoop sobre Máquina Virtual CentOS
Arquitectura utilizada	1 nodo maestro + 2 nodos trabajadores
Framework de procesamiento 1	Hadoop MapReduce
Framework de procesamiento 2	Apache Spark
Dataset de prueba 1	ventas_200MB.csv
Dataset de prueba 2	ventas_1GB.csv
Dataset de prueba 3	dataset_5GB.csv
Sistema de almacenamiento	Hadoop Distributed File System (HDFS)
Administrador de recursos	YARN
Número total de ejecuciones	12 pruebas experimentales

1) Preparación de los datasets

Inicialmente, los conjuntos de datos utilizados en las pruebas experimentales fueron almacenados dentro del sistema distribuido de archivos HDFS en cada uno de los escenarios evaluados: clúster Hadoop desplegado mediante contenedores Docker y clúster Hadoop implementado sobre máquinas virtuales con sistema operativo CentOS.

Esta etapa permitió garantizar que ambos ambientes de ejecución trabajaran bajo las mismas condiciones de entrada, utilizando los mismos archivos y volúmenes de información antes de iniciar los procesos de procesamiento distribuido.

La ejecución del comando “`hdfs dfs -ls /datasets`” permitió comprobar la disponibilidad de los archivos dentro del directorio de almacenamiento distribuido en ambos escenarios experimentales, validando la presencia de los tres conjuntos de datos utilizados.

Fig. 5. Verificación de datasets almacenados en HDFS en los escenarios Docker y Máquina Virtual antes de la ejecución experimental

2) Ejecución de cargas de procesamiento distribuido

Después de validar la disponibilidad de los datos, se ejecutaron los procesos correspondientes utilizando Hadoop MapReduce y Apache Spark. Cada framework fue probado con los tres tamaños de dataset definidos, tanto en el escenario Docker como en el escenario basado en máquinas virtuales.

La combinación de escenarios, frameworks y datasets generó un total de doce ejecuciones experimentales, distribuidas de la siguiente manera:

Tabla 3. Distribución de ejecuciones experimentales

Ambiente	Framework	Dataset	Ejecución
Docker	MapReduce	200 MB	Prueba 1
Docker	MapReduce	1 GB	Prueba 2
Docker	MapReduce	5 GB	Prueba 3
Docker	Spark	200 MB	Prueba 4
Docker	Spark	1 GB	Prueba 5
Docker	Spark	5 GB	Prueba 6
Máquina Virtual	MapReduce	200 MB	Prueba 7
Máquina Virtual	MapReduce	1 GB	Prueba 8

Máquina Virtual	MapReduce	5 GB	Prueba 9
Máquina Virtual	Spark	200 MB	Prueba 10
Máquina Virtual	Spark	1 GB	Prueba 11
Máquina Virtual	Spark	5 GB	Prueba 12

2.5 Métricas de evaluación

Con el propósito de analizar el rendimiento del ecosistema Hadoop en los escenarios experimentales definidos, se establecieron métricas orientadas a evaluar el comportamiento del procesamiento distribuido bajo diferentes condiciones de ejecución. Estas métricas permitieron obtener información relacionada con el tiempo requerido para completar las tareas, el consumo de recursos computacionales y la utilización de los recursos administrados por el gestor YARN.

Las métricas seleccionadas fueron aplicadas de manera uniforme en ambos escenarios experimentales: clúster Hadoop desplegado mediante Docker y clúster Hadoop implementado sobre máquinas virtuales CentOS. Asimismo, fueron evaluadas para los frameworks Hadoop MapReduce y Apache Spark utilizando los datasets definidos previamente.

Tabla 4. Métricas utilizadas para la evaluación del rendimiento

Métrica	Descripción	Fuente de información
Tiempo de ejecución	Duración total del procesamiento de cada tarea	Consola Hadoop / registros de ejecución
Consumo de CPU	Nivel de utilización del procesador durante la ejecución	Monitor del sistema / YARN
Consumo de memoria	Cantidad de memoria utilizada por los procesos Hadoop	YARN / sistema operativo
Memory Seconds	Memoria acumulada utilizada por una aplicación	YARN
VCore Seconds	Uso acumulado de recursos de procesamiento virtual	YARN
Contenedores utilizados	Cantidad de contenedores asignados durante la ejecución	YARN
Estado de ejecución	Resultado final del procesamiento (éxito o fallo)	YARN

Para la recopilación de estas métricas se utilizó la información generada por el administrador de recursos YARN, mediante los reportes de ejecución de cada aplicación Hadoop. La consulta de información fue realizada mediante comandos del ecosistema Hadoop, permitiendo obtener los valores asociados a cada prueba experimental.

Fig. 7. Obtención de métricas de ejecución mediante YARN

Tabla 5. Relación entre variables experimentales y métricas evaluadas

Factor experimental	Métricas analizadas
Docker vs Máquina Virtual	Tiempo, CPU, memoria, recursos YARN
MapReduce vs Spark	Tiempo, Memory Seconds, VCore Seconds
Dataset 200MB	Uso inicial de recursos
Dataset 1GB	Escalabilidad intermedia
Dataset 5GB	Capacidad ante cargas mayores

2.6 Análisis comparativo de resultados

Los datos obtenidos durante las ejecuciones experimentales serán analizados con el propósito de comparar el rendimiento del ecosistema Hadoop en los escenarios basados en Docker y máquinas virtuales CentOS.

El análisis considerará las métricas recopiladas durante las pruebas, incluyendo el tiempo de ejecución, consumo de recursos computacionales y valores proporcionados por el administrador de recursos YARN. La comparación será realizada considerando los diferentes frameworks evaluados (Hadoop MapReduce y Apache Spark) y los distintos tamaños de datasets utilizados.

Para garantizar la validez de la comparación, los resultados serán analizados bajo las mismas condiciones experimentales, considerando el mismo dataset, framework y configuración de ejecución en ambos ambientes de despliegue.

Finalmente, los datos obtenidos serán representados mediante tablas comparativas y gráficos estadísticos, permitiendo identificar las diferencias de rendimiento, comportamiento de los recursos utilizados y capacidad de escalabilidad del sistema frente al incremento del volumen de información.

3 RESULTADOS

En esta sección se presentan los resultados obtenidos después de ejecutar las pruebas experimentales definidas en la metodología. Los datos recopilados permiten analizar el comportamiento del ecosistema Hadoop en los escenarios Docker y Máquina Virtual CentOS, considerando diferentes frameworks de procesamiento y tamaños de datasets.

3.1 Análisis del Tiempo de Ejecución

3.2 Análisis del Consumo de Memoria y CPU (VCore-segundos)

3.2 Análisis de Contenedores y Escalabilidad

4 DISCUSIÓN

5 CONCLUSIONES

CONTRIBUCIÓN DE LOS AUTORES

FINANCIAMIENTO

ÉTICA

DISPONIBILIDAD DE LOS DATOS

AGRADECIMIENTOS

CONFLICTO DE INTERESES

REFERENCIAS

- [1] P. Liu y J. Guitart, "Performance characterization of containerization for HPC workloads on InfiniBand clusters: an empirical study," *Cluster Computing*, vol. 25, pp. 847–868, 2022, doi: 10.1007/s10586-021-03460-8.
- [2] W.-C. Shih, C.-T. Yang, R. Ranjan, y C.-I. Chiang, "Implementation and evaluation of a container management platform on Docker: Hadoop deployment as an example," *Cluster Computing*, vol. 24, pp. 2851–2868, 2021, doi: 10.1007/s10586-021-03337-w.
- [3] A. Bhardwaj, K. Pandey, R. Sharma, L. Deewan, y U. Gupta, "Comparative Performance Analysis of Docker Containers and Virtual Machines: A Comprehensive Evaluation," 2025 International Conference on Next Generation Information Systems Engineering (NGISE), Ghaziabad, Delhi (NCR), India, Mar. 28–29, 2025, pp. 1–6, doi: 10.1109/NGISE64126.2025.11085383.
- [4] A. M. Potdar, N. D. G., S. Kengond, y M. M. Mulla, "Performance Evaluation of Docker Container and Virtual Machine," *Procedia Computer Science*, vol. 171, pp. 1419–1428, 2020, doi: 10.1016/j.procs.2020.04.152.
- [5] C. Calcaterra, A. Carmenini, A. Marotta, U. Bucci, y D. Cassioli, "MaxHadoop: An Efficient Scalable Emulation Tool to Test SDN Protocols in Emulated Hadoop Environments," *Journal of Network and Systems Management*, vol. 28, pp. 1610–1638, 2020, doi: 10.1007/s10922-020-09552-x.
- [6] S. Lim y D. Park, "Improving Hadoop MapReduce Performance in Heterogeneous Single-Board Computer Clusters," *Future Generation Computer Systems*, vol. 160, pp. 752–766, Nov. 2024, doi: 10.1016/j.future.2024.06.025.
- [7] A. El Yazidi, M. S. Azizi, Y. Benlachmi, y M. L. Hasnaoui, "Apache Hadoop-MapReduce Latency in YARN Framework," *Procedia Computer Science*, vol. 184, pp. 803–808, 2021, doi: 10.1016/j.procs.2021.03.100.
- [8] Y. Li y X. Hei, "Performance Optimization of Computing Task Scheduling Based on the Hadoop Big Data Platform," *Neural Computing and Applications*, vol. 37, pp. 8181–8192, 2025, doi: 10.1007/s00521-022-08114-3.
- [9] M. Saadoon, S. H. Ab Hamid, H. Sofia, H. Altartur, N. Nasuha, Z. H. Azizul, A. A. Sani, y A. Asemi, "Experimental Analysis of Hadoop MapReduce: A More Detailed Study of Fault Detection and Recovery Techniques," *Sensors*, vol. 21, no. 11, p. 3799, 2021, doi: 10.3390/s21113799.
- [10] D. Huang, J. Wang, Q. Liu, N. Xiao, H. F. Wu, y J. Yin, "Improving Proportional I/O Sharing in Containerized Big Data File Systems," *IEEE Transactions on Computers*, vol. 70, no. 12, pp. 2083–2097, Dec. 2021, doi: 10.1109/TC.2020.3037078.
-